

Roadmap del TP de Microservicios

Guía paso a paso para el trabajo práctico de E-Commerce con .NET Core 10 — desde crear el repositorio en grupo hasta llegar a la defensa, en el orden que sugiere la cátedra.

Modalidad Grupal · 3 integrantes **Duración** 6 a 7 semanas

Stack C# · .NET Core 10 · ASP.NET Core

Servicios Products · Users · Cart · Orders · Notifications

Mapa general

Guía por integrante

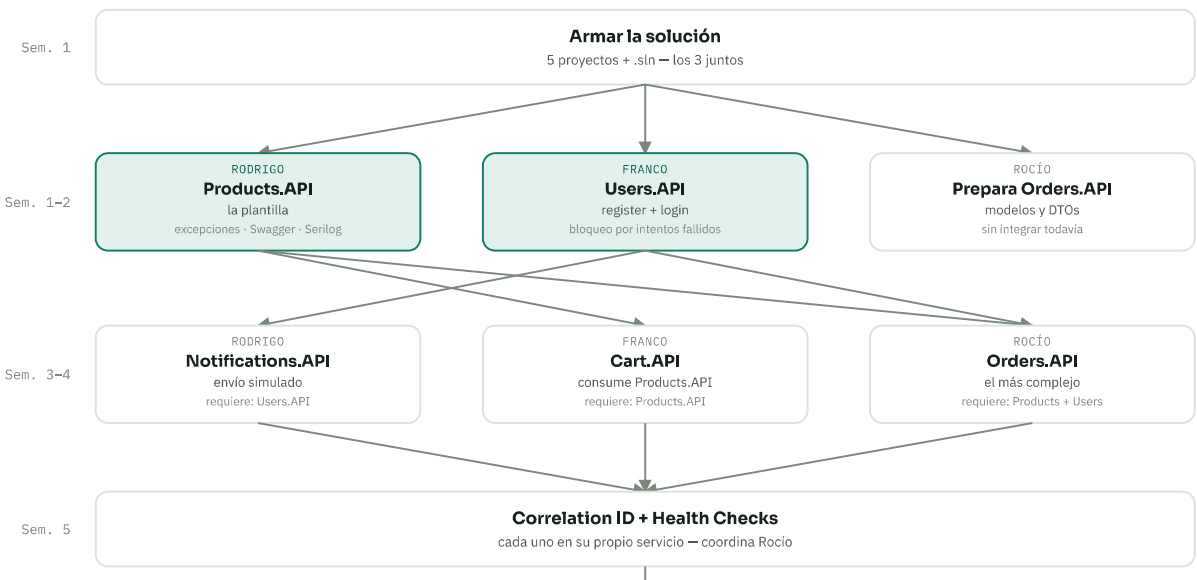
00 Antes de arrancar

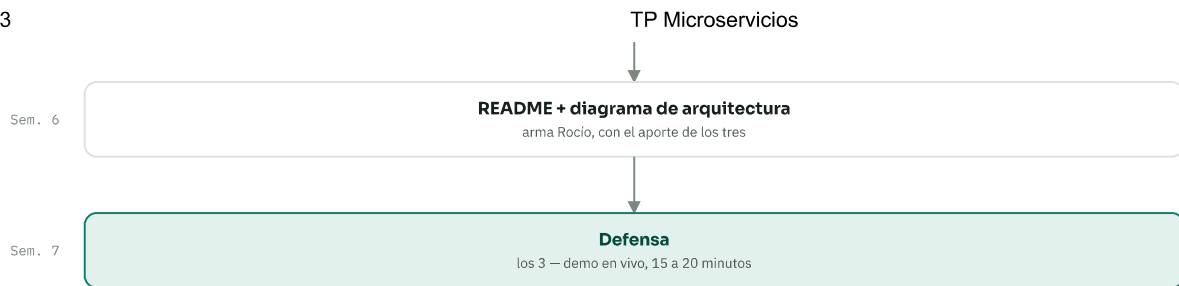
01 Crear el repo

02

Mapa general: qué hace cada uno, y en qué orden

Cada columna es una persona y cada fila un momento del proyecto. Las flechas van de un servicio a otro cuando el segundo *necesita* que el primero ya esté terminado — si no hay flecha entre dos cajas de la misma fila, se pueden hacer en paralelo sin pisarse.





Products.API y Users.API (destacados) son los dos servicios que más bloquean al resto — por eso arrancan primero y en paralelo. Una vez listos, Notifications.API, Cart.API y Orders.API también se pueden hacer los tres en simultáneo, cada uno con su propia flecha de "requiere". El tiempo por semana es orientativo sobre 6-7 semanas totales.

Guía por integrante

El mismo trabajo, pero visto desde la silla de cada uno: qué hacés primero, cuándo tenés que frenar a esperar a un compañero, y en qué momento lo que vos terminaste destranca a otro. Los recuadros verdes son "aviso al grupo, esto habilita algo"; los ámbar son "acá frenás a esperar".

Rodrigo

Products.API + Notifications.API

1

Preparación y estructura del repo, junto con Franco y Rocío — instalar herramientas, crear el repo, agregar colaboradores, armar la solución con los 5 proyectos (fases 00 a 03 de la guía).

2

Arrancás de una con Products.API completo: modelo, DTOs, los 5 endpoints, catálogo PRD-001 a PRD-005, excepciones, los tres IExceptionHandler, Swagger y Serilog. No dependés de nadie — es el primer servicio del TP.

3

ESTO HABILITA A OTROS

En cuanto mergeás Products.API a main, avisale al grupo. Franco necesita que esté listo para arrancar Cart.API, y Rocío lo necesita para Orders.API.

4

ACÁ FRENÁS

Notifications.API depende de que Users.API esté terminado. Mientras Franco lo termina, revisale el Pull Request, o ayudá a Rocío a modelar Orders.API — no te quedes esperando sin hacer nada.

5

Cuando Users.API esté mergeado, arrancás **Notifications.API**: los 2 endpoints, catálogo NTF-001 a NTF-004, validando contra Users.API que el usuario destinatario exista.

6

Agregás Correlation ID y Health Checks a tus dos servicios (Products.API y Notifications.API), siguiendo el patrón que coordina Rocío.

7

Le pasás a Rocío tu tabla de puertos y códigos de error de Products y Notifications para el README, y preparás tu parte de la demo — pero repasá también Users, Cart y Orders antes de la defensa.

Franco

Users.API + Cart.API

1

Preparación y estructura del repo, junto con Rodrigo y Rocío (fases 00 a 03 de la guía). Aceptá la invitación como colaborador apenas te llegue.

2

Arrancás de una con Users.API, en paralelo con Rodrigo mientras hace Products.API — no dependés de nadie. Register y login, catálogo USR-001 a USR-006, y prestá especial atención a la lógica de bloqueo por intentos fallidos (USR-004) vs. bloqueo manual por fraude (USR-005).

3

ESTO HABILITA A OTROS

En cuanto mergeás Users.API, avisale al grupo. Rodrigo lo necesita para Notifications.API y Rocío lo necesita para Orders.API

Notifications API, y Rocío lo necesita para Orders API.

4

ACÁ FRENÁS

Cart.API depende de que Products.API esté terminado. Mientras Rodrigo lo termina, revisale el Pull Request, o ayudá a Rocío con Orders.API.

5

Cuando Products.API esté mergeado, arrancás **Cart.API**: los 5 endpoints, catálogo CRT-001 a CRT-005, consumiendo Products.API vía `IHttpClientFactory` para validar stock antes de agregar o actualizar un item.

6

Agregás Correlation ID y Health Checks a tus dos servicios (Users.API y Cart.API), siguiendo el patrón que coordina Rocío.

7

Le pasás a Rocío tu tabla de puertos y códigos de error de Users y Cart para el README, y preparás tu parte de la demo — pero repasá también Products, Orders y Notifications antes de la defensa.

Rocío

Orders.API + Correlation ID/Health + README

1

Preparación y estructura del repo, junto con Rodrigo y Franco (fases 00 a 03 de la guía).

2

Mientras Rodrigo y Franco arrancan Products.API y Users.API en paralelo, vos **preparás Orders.API sin integrar todavía**: el modelo `Order` / `OrderItem`, los DTOs, el catálogo ORD-001 a ORD-007, y el mapa de transiciones de estado válidas (Pendiente → Confirmada → Enviada → Entregada, o Cancelada). Nada de esto depende de que los otros dos terminen, así que podés avanzar en paralelo.

3

Sos la única de los tres sin bloqueo en esta etapa — aprovechá para revisar los Pull Requests de Products.API y Users.API a medida que Rodrigo y Franco los van abriendo. Además de ayudar al equipo, llegás a la integración conociendo ya el código de los dos servicios de los que depende Orders.

4

ACÁ FRENÁS

Orders.API es el único servicio que depende de **dos** compañeros a la vez: no conectás nada de verdad hasta que Products.API y Users.API estén los dos mergeados a main.

5

Con los dos mergeados, integrás **Orders.API** de verdad: llamadas HTTP a Users.API y Products.API para validar usuario, productos y stock (ORD-003 a ORD-005), cálculo automático del total, y las transiciones de estado (ORD-006 cuando es inválida).

6

COORDINÁS AL GRUPO

Con los cinco servicios funcionando, coordinás el Correlation ID y los Health Checks: definís el patrón (podés tomar como base lo que Rodrigo ya armó en Products.API) y confirmás que Rodrigo y Franco lo apliquen cada uno en los suyos.

7

Armás el **README final** juntando las tablas de puertos y errores que te pasan Rodrigo y Franco, el diagrama de arquitectura y las capturas de Swagger. Coordinás también el ensayo de la defensa entre los tres.

El camino completo, en 12 pasos

Cada fase te dice qué hacer, en qué orden y con qué comandos. Seguilo tal cual la

primera vuelta (Products.API antes que nada, porque es la plantilla que después se replica) y a partir de la fase 05 repartan el trabajo entre los tres integrantes.

00

Antes de arrancar

Lo que necesita tener instalado **cada integrante** antes del primer commit, según la Guía de Git de la cátedra.

- ☐ Git instalado y configurado (`git config --global user.name` / `user.email`)
- ☐ Cuenta de GitHub creada, misma que van a usar los tres
- ☐ .NET Core 10 SDK instalado (verificar con `dotnet --version`)
- ☐ Visual Studio (o VS Code + extensión de C#)
- ☐ GitHub Desktop instalado si prefieren interfaz gráfica en vez de terminal para los commits

ANTES DEL CÓDIGO

Definan quién crea el repo, un canal de chat para avisos rápidos ("subí X", "voy a mergear Y") y quién será el que comparte el repo con el docente. Son decisiones de dos minutos que evitan pisadas después.

01

Crear el repositorio `git init`

Uno del grupo crea el repositorio en GitHub (público, o privado compartido con el docente) y arma el esqueleto que pide la cátedra en la sección "Estructura del Proyecto".

```
# 1. En GitHub: New repository → nombre "ECommerce" → Add .gitignore:

# 2. Clonarlo en tu máquina
git clone https://github.com/tu-usuario/ECommerce.git
cd ECommerce

# 3. Crear la solución y las carpetas base
dotnet new sln -n ECommerce
mkdir src docs

# 4. Primer commit
git add .
git commit -m "Estructura inicial del repositorio"
git push origin main
```

- ☐ Agregar a los otros dos *Collaborators* (Settings → Collaborators) o al docente si el repo va a quedar privado
- ☐ Confirmar `.gitignore` excluye `bin/` , `obj/` , `*.user` y las `logs/` carpetas que el
- ☐ Crear un `README.md` vacío por ahora — se completa en la fase 10

02

Definir la estrategia de ramas

La rama `main` queda protegida: **nadie commitea directo ahí**. Todo entra por Pull Request. Usen una branch por servicio o por tarea.

Convención

`feature/products-api` ,
`feature/users-login` ,
`fix/orders-stock-check`

Regla de oro

Dos personas nunca trabajan en la misma branch al mismo tiempo, ni tocan el mismo microservicio en paralelo sin avisar.

FLUJO DIARIO (REPETIR SIEMPRE)

```
git checkout main
git pull origin main          # 1. actualizarte primero
git checkout -b feature/mi-tarea # 2. tu propia rama
git add . && git commit -m "Agrego X" # 3. commits chicos y frecuentes
git push -u origin feature/mi-tarea # 4. subir
# 5. Abrir Pull Request en GitHub → pedir revisión a un compañero
```

- ☐ Antes de tocar código: `git pull origin main`
- ☐ Commits chicos con mensajes en presente ("Agrego validación de stock", no "arreglos varios")
- ☐ Un compañero revisa cada PR antes de aprobar el merge — así todos conocen el código de todos, algo que van a necesitar en la defensa
- ☐ Si aparece `<<<<<<` , decidir con quién generó el otro un cambio, borrar los marcadores y `git push --force` sobre una rama compartida. buscar

REGLA DE LA CÁTEDRA: ROTAR QUIÉN COMMITTEA

Piden que cada commit quede a nombre de un integrante distinto, para poder verificar que los tres participaron. La forma más simple de cumplirlo es apoyarse en el punto anterior — **commits chicos y frecuentes** — y turnarse en quién aprieta el botón, aunque estén programando juntos.

Si cada uno trabaja desde su propia notebook (lo más común): alcanza con que cada persona haga sus propios commits de lo que va haciendo, sin que uno solo del grupo termine "commiteando por los demás". Verifiquen que `git config user.name` / `user.email` en cada máquina coincida con el mail de su cuenta de GitHub.

Si están programando juntos en una sola máquina (mob/pair programming), roten quién es el "driver" que commitea en cada paso, o atribuyan el commit a quien corresponda con:

```
git commit -m "Agrego validación de stock" --author="Nombre Apell
```

El email tiene que estar verificado en la cuenta de GitHub de esa persona (Settings → Emails), si no el commit no se le va a atribuir. Antes de entregar, revisen *Insights* → *Contributors* en GitHub para confirmar que la distribución entre los tres se ve pareja.

03**Armar la solución y los 5 proyectos**

Se crean los cinco Web API y se agregan todos a la misma `.sln`, tal como pide la consigna.

```
cd src
dotnet new webapi -n Products.API -o Products.API
dotnet new webapi -n Users.API -o Users.API
dotnet new webapi -n Orders.API -o Orders.API
dotnet new webapi -n Cart.API -o Cart.API
dotnet new webapi -n Notifications.API -o Notifications.API
cd ..

dotnet sln add src/Products.API/Products.API.csproj ^
src/Users.API/Users.API.csproj ^
src/Orders.API/Orders.API.csproj ^
src/Cart.API/Cart.API.csproj ^
src/Notifications.API/Notifications.API.csproj
```


Dentro de **cada** proyecto van a necesitar las mismas carpetas (la cátedra las pide explícitamente):

```
Products.API/
├── Controllers/
├── Models/           # entidades del dominio
├── DTOs/             # request y response
├── Services/         # lógica de negocio
├── Exceptions/       # NotFoundException, BusinessException...
├── ExceptionHandlers/ # un IExceptionHandler por tipo
├── logs/             # salida de Serilog
└── Program.cs
```

Paquetes NuGet a instalar en cada servicio:

```
dotnet add package Swashbuckle.AspNetCore
dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.Console
dotnet add package Serilog.Sinks.File
dotnet add package Microsoft.Extensions.Diagnostics.HealthChecks
```

SOBRE LA PERSISTENCIA

La consigna dice que la persistencia la provee la cátedra como librería — ustedes solo invocan sus métodos desde la capa de `Services/`. No hace falta levantar una base de datos propia ni configurar un ORM.

Commit y push de esta estructura antes de seguir — es la base sobre la que van a trabajar los tres en paralelo.

04

Products.API — la plantilla bloqueante

Es el **primer** servicio a completar y no conviene paralelizarlo: una vez que funciona entero, su estructura se copia tal cual al resto. Que lo haga una sola persona (o los tres juntos) evita rehacer trabajo.

☐ Modelo `Product` (Id, Nombre, Descripción, Precio, Stock, Categoría, FechaCreación) + DTOs de request/response

☐ Los 5 endpoints: `GET /api/products`, `GET /api/products/{id}`, `POST /api/products`, `PUT /api/products/{id}`, `DELETE /api/products/{id}`

endpoints `/api/products``/api/products/{id}``/api/``/api/`

REST:

- ☐ Catálogo de errores propio como constantes o enum: `PRD-001` a `PRD-005`
- ☐ Excepciones `ErrorCode` : `NotFoundException` , `BusinessRuleException` de dominio con
- ☐ `ExceptionHandler` por tipo, registrados del más específico al genérico
- ☐ Swagger `/swagger` con XML comments, tags, y ejemplos de response de éxito y de error
- ☐ Serilog configurado — consola legible + archivo JSON en `logs/`
- ☐ `/health` , `/health/ready` , `/health/live`

PATRÓN DE PROGRAM.CS (APÉNDICE B DEL TP)

```
// Serilog SIEMPRE primero en Program.cs, antes que cualquier otro
builder.Host.UseSerilog((ctx, cfg) => cfg
    .WriteTo.Console()
    .WriteTo.File("logs/log-.json", rollingInterval: RollingIntervalDay));
```

```
// Handlers en orden de especificidad: primero los específicos, último el genérico
builder.Services.AddExceptionHandler<NotFoundExceptionHandler>();
builder.Services.AddExceptionHandler<BusinessRuleExceptionHandler>();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();
// ...
app.UseExceptionHandler();
```

```
// Excepción de dominio con ErrorCode propio
public class NotFoundException(string errorCode, string message)
{
    public string ErrorCode { get; } = errorCode;
}

// Se lanza desde la capa de Services — el handler arma la respuesta
if (product is null)
    throw new NotFoundException("PRD-001", "Producto no encontrado");
```

No pasen a la fase 05 hasta que Products.API responda éxito y error en los 5

endpoints, con Swagger navegable y logs escribiéndose en `logs/`

endpoints, con Swagger navegable y logs escribiéndose en `logs/`.

05

Users.API

se puede paralelizar desde acá

Copien la estructura de Products.API (Controllers, Exceptions, ExceptionHandlers, Serilog, Swagger, health checks) y adaptenla. A partir de esta fase, los tres integrantes pueden trabajar en servicios distintos al mismo tiempo, cada uno en su propia branch.

- ☐ `POST /api/users/register` y `POST /api/users/login`
- ☐ El `PasswordHash` nunca se incluye en ninguna response
- ☐ Lógica de bloqueo: `IntentosFallidos` consecutivos `3` → `Activo = false` (`USR-004`), y `USR-005` distinguirlo del bloqueo manual por fraude (`USR-006`)
- ☐ Catálogo `USR-001` a `USR-006`

06

Cart.API

depende de Products.API

El carrito llama a Products.API por HTTP para validar stock antes de agregar o actualizar un item.

- ☐ Registrar `IHttpClientFactory` apuntando a la URL de Products.API
- ☐ `GET /api/cart/{userId}`, `POST/PUT/DELETE .../items/{productId}`, `DELETE /api/cart/{userId}`
- ☐ Catálogo `CRT-001` a `CRT-005` (stock insuficiente = `CRT-003`)

RECOMENDACIÓN DE LA CÁTEDRA

Verificar siempre el HTTP status de la respuesta de Products.API antes de procesar el body. Si Products.API devuelve 404, no asuman que el body trae un producto válido.

07

Orders.API

depende de Users y Products

Es el servicio con más validaciones cruzadas: valida que el usuario y los

Es el servicio con más validaciones cruzadas. Valida que el usuario y los productos existan, y que haya stock, antes de crear la orden.

- ☐ `ORD-003` usuario inexistente, `ORD-004` producto inexistente, `ORD-005` stock insuficiente (422)
- ☐ Transiciones de estado válidas:

Pendiente
→ Confirmada
→ Enviada
→ Entregada

, `Cancelada` o . Una transición no permitida (ej. volver de "Entregada" a "Pendiente") devuelve `ORD-006` (409)
- ☐ `Total` se calcula automáticamente a partir de los items, no lo manda el cliente

08

Notifications.API

servicio de soporte

El más simple de los cinco — registra y simula el envío de notificaciones, no requiere integrarse con un proveedor real de email/SMS.

- ☐ `POST /api/notifications/send` y `GET /api/notifications/{userId}`
- ☐ Valida que el usuario destinatario exista contra Users.API (`NTF-001`)

09

Transversal: Correlation ID + Health Checks

Con los cinco servicios funcionando, se juntan de nuevo los tres para lo que atraviesa a todos por igual. Buen momento para hacerlo en una branch conjunta y revisarlo entre todos.

- ☐ Generar `X-Correlation-Id` único por request entrante (si no viene, se crea uno nuevo)
- ☐ Propagarlo en cada llamada HTTP saliente entre microservicios (Cart→Products, Orders→Users/Products, etc.)
- ☐ Incluirlo en todos los logs de Serilog y como campo extra en las respuestas de error
- ☐ Confirmar `/health` , `/health/ready` , `/health/live` respondiendo en los 5 servicios

10

Documentación y checklist de entrega

Lo que la cátedra pide explícitamente en "Condiciones de Entrega".

- ☐ **README.md** con: cómo levantar cada microservicio, tabla de puertos, y tabla completa de códigos de error
- ☐ Diagrama de arquitectura (imagen o ASCII en el README) mostrando los 5 servicios y quién le habla a quién
- ☐ Capturas de Swagger UI mostrando al `errorCode` y `errorMessage` por menos un ejemplo de response de servicio error con
- ☐ Repositorio Git público, o compartido con el docente si es privado
- ☐ Revisar *Insights* → en GitHub: los commits de los tres integrantes tienen que *Contributors* verse repartidos, no concentrados en uno solo